

1) Merkle Tree and Merkle Proof

What is a Merkle Tree?

A **Merkle Tree** is a hash tree used to summarize all transactions inside a block.

Instead of keeping one huge flat list and checking every transaction one by one, Bitcoin:

- hashes each transaction,
- pairs those hashes,
- hashes the pairs,
- keeps repeating this upward,
- until only **one final hash** remains.

That final top hash is called the **Merkle Root**.

So a Merkle Tree is basically:

a tree of hashes whose top hash summarizes all block transactions.

Why is it needed?

Because a block may contain many transactions, and Bitcoin needs:

- a compact summary of all of them
- a fast way to prove one transaction is included
- a way to detect tampering

If even one transaction changes:

- its transaction hash changes
- parent hash changes
- hashes above it change
- Merkle root changes

So Merkle root helps maintain integrity.

Structure of a Merkle Tree

Suppose a block has 4 transactions:

- Tx1
- Tx2
- Tx3
- Tx4

First hash each transaction:

- $H1 = \text{Hash}(\text{Tx1})$
- $H2 = \text{Hash}(\text{Tx2})$
- $H3 = \text{Hash}(\text{Tx3})$
- $H4 = \text{Hash}(\text{Tx4})$

Then combine in pairs:

- $H12 = \text{Hash}(H1 \parallel H2)$
- $H34 = \text{Hash}(H3 \parallel H4)$

Then combine again:

- $\text{Root} = \text{Hash}(H12 \parallel H34)$

That final Root is the **Merkle Root** stored in the block header.

What if number of transactions is odd?

If there is an odd number of transaction hashes at some level, Bitcoin duplicates the last one before hashing upward.

Example:

- H1, H2, H3

Then:

- $H12 = \text{Hash}(H1 \parallel H2)$
- $H33 = \text{Hash}(H3 \parallel H3)$

Then:

- $\text{Root} = \text{Hash}(H12 \parallel H33)$

This ensures every level can still be paired.

What is a Merkle Proof?

A **Merkle Proof** is a compact proof that a transaction is included in a block.

Instead of downloading all transactions in the block, you only need:

- the transaction's own hash
- the sibling hashes along the path to the root

Then you recompute upward and see whether you get the correct Merkle root.

If yes, the transaction is proven to be in that block.

Example of Merkle Proof

Suppose you want to prove **Tx2** is in the block.

You need:

- $H(Tx2)$
- sibling hash $H(Tx1)$
- sibling parent hash $H34$

Then compute:

1. $H12 = \text{Hash}(H1 \parallel H2)$
2. $\text{Root} = \text{Hash}(H12 \parallel H34)$

If computed Root matches the block header's Merkle root, then Tx2 is in the block.

Why is this important?

Because this allows:

- **efficient inclusion proof**
- **lightweight verification**
- **tamper detection**

You do **not** need the whole block.

$O(n)$ vs $O(\log n)$

Without a Merkle tree, proving inclusion may require checking all transactions:

- time complexity: **$O(n)$**

With a Merkle tree, you only need the path from leaf to root:

- time complexity: **$O(\log n)$**

That is a huge efficiency improvement.

SPV connection

Merkle Proof is used in **SPV (Simplified Payment Verification)**.

SPV clients do not download the full blockchain. They only download:

- block headers
- relevant Merkle proofs

So they can verify a payment without becoming a full node.

What sir can ask

Sir can ask:

- define Merkle tree
 - define Merkle proof
 - compute a Merkle root from given transaction hashes
 - prove one transaction belongs to a block
 - compare $O(n)$ vs $O(\log n)$
 - explain why Merkle root changes if one transaction changes
-

2) Bitcoin Transaction Signature Construction and Verification

This is a very important topic.

Why does Bitcoin need signatures?

Bitcoin needs signatures so that only the rightful owner can spend the coins.

If Alice owns a UTXO, then only Alice should be able to authorize spending it. That authorization is done with a **digital signature** created using Alice's **private key**.

What is signed in a Bitcoin transaction?

Conceptually, the signer signs the transaction data that matters, not just random text.

In simple course style, think of it as signing:

- payer
- payee
- amount
- referenced input/output info
- relevant transaction fields

More properly:

Bitcoin signs the **transaction data (excluding the signature field itself)** after serialization and hashing, according to the sighash rules.

So the logic is:

1. Construct transaction message
 2. Hash the transaction message
 3. Sign that hash using private key
 4. Put signature into unlocking data
-

Signature construction

Step 1: Create transaction data

The transaction includes:

- referenced input UTXO
- output values
- output locking scripts
- other transaction fields

Step 2: Hash the transaction

Bitcoin hashes the transaction message.

Step 3: Sign using private key

The payer uses their private key to sign the hash.

Bitcoin legacy signatures use **ECDSA**.

An ECDSA signature consists of:

- **r**
- **s**

So the signature is basically a pair (**r**, **s**) plus sighash type information.

Step 4: Put signature into transaction input

In legacy transactions, the signature is placed in **ScriptSig**.

In SegWit transactions, it goes in the **witness**.

Signature verification

When a node verifies the transaction, it checks:

1. Is the public key provided?
2. Does the public key match the expected lock condition?
3. Does the signature verify correctly under that public key?
4. Was the signed transaction message the correct one?

If yes, then the spender is authorized.

Why hashing before signing?

Because signing the raw whole transaction directly is not the cleanest approach.

Hashing first gives:

- fixed-length digest
- efficient signing
- integrity guarantee

If the transaction changes even slightly, the hash changes, so the signature no longer verifies.

ECDSA format reminder

The lecture also showed:

- signature = (r, s)
- encoded in DER-like format
- includes sighash byte

You do not usually need to derive ECDSA math in the exam, but you should know:

- Bitcoin uses ECDSA in legacy context
 - signature proves authorization
 - signature is verified using public key
-

What sir can ask

Sir can ask:

- how a Bitcoin transaction signature is constructed
 - why Bitcoin signs the hash, not plain message
 - what is verified during signature verification
 - what are r and s
 - what is the role of private key and public key
 - what does a valid signature prove
-

3) Bitcoin UTXO Unlocking

This is one of the most important exam topics.

What does “unlocking a UTXO” mean?

A UTXO is an **Unspent Transaction Output**.

That output has:

- some value
- a locking condition (`scriptPubKey`)

When someone wants to spend it, they must provide the correct unlocking data.

That process is called **UTXO unlocking**.

Basic idea

The old output says:

“You can spend me only if you satisfy this condition.”

The new spending input says:

“Here is the proof that I satisfy it.”

Then Bitcoin executes:

- unlocking script / witness
- together with locking script

If result is **TRUE**, the UTXO is unlocked and spent.

Parent and child transaction

- **Parent transaction** creates the UTXO
- **Child transaction** spends that UTXO

So unlocking always happens when a child transaction tries to spend a parent’s output.

In legacy P2PKH

Parent output contains:

`scriptPubKey`

Usually something like:

- OP_DUP
- OP_HASH160
- expected pubKeyHash
- OP_EQUALVERIFY
- OP_CHECKSIG

Child input contains:

`ScriptSig`

- signature
- public key

Then the script executes.

What is checked during unlocking?

Bitcoin checks two main things:

1. Public key ownership check

Does the provided public key hash to the same value that the UTXO was locked to?

If no → fail

If yes → continue

2. Signature check

Does the provided signature verify correctly under that public key for this transaction?

If no → fail

If yes → success

So unlocking proves:

- correct key identity
 - valid authorization
-

Stack execution idea

Bitcoin Script is stack-based.

For P2PKH, the flow is conceptually:

1. Push signature
2. Push public key
3. Duplicate public key
4. Hash one copy
5. Compare with expected pubkey hash
6. Verify signature

If all checks succeed:

final result = TRUE

That means the output is unlocked.

In SegWit

In SegWit:

- `scriptSig` is empty
- signature + public key are stored in **witness**
- the logic still checks ownership and signature validity

So the location changes, but the basic unlocking idea remains.

What sir can ask

Sir can ask:

- explain UTXO unlocking

- parent vs child transaction
 - how ScriptSig and ScriptPubKey work together
 - simulate stack execution
 - explain why final TRUE means valid spend
 - compare legacy unlocking vs SegWit unlocking
-

4) Proof-of-Work and Mining (target hash and nonce)

What is Proof-of-Work?

Proof-of-Work (PoW) is Bitcoin's consensus mechanism.

It means miners must do computational work to create a valid block.

They do this by hashing the block header repeatedly until they find a hash that satisfies the required difficulty condition.

What is mining?

Mining is the process of:

- collecting transactions
- building a candidate block
- repeatedly changing the nonce
- hashing the block header
- trying to get a valid hash under the target

So mining is basically a huge trial-and-error search.

Target hash

The **target hash** or difficulty target defines how hard mining is.

A valid block header hash must be:

- below the target
or equivalently in simplified classroom terms:
- begin with enough zeros / satisfy difficulty condition

The lower the target, the harder mining becomes.

Nonce

The **nonce** is the value miners keep changing.

Why?

Because if block contents are mostly fixed, miners need some adjustable field to make the header hash come out differently on each attempt.

So miners do:

- nonce = 1 → hash
 - nonce = 2 → hash
 - nonce = 3 → hash
- and so on

Until a valid hash is found.

Why is this secure?

Because finding a valid hash is hard and costly.

This prevents cheap rewriting of blocks and supports decentralized consensus.

Difficulty adjustment

Bitcoin adjusts difficulty every **2016 blocks** so that average block generation stays around **10 minutes**.

If miners become faster overall, difficulty increases.

If miners become slower overall, difficulty decreases.

What sir can ask

Sir can ask:

- define Proof-of-Work
 - define mining
 - explain target hash
 - explain nonce
 - simulate mining if given target and sample hashes
 - explain why changing nonce changes hash
 - explain why PoW helps consensus
-

5) UTXO Database

What is the UTXO database?

The UTXO database stores all outputs that are currently:

- valid
- unspent
- spendable

In Bitcoin Core, this is kept in **chainstate**.

This database helps nodes validate transactions quickly.

Why is it needed?

Because UTXOs are created across many blocks in blockchain history.

Without a UTXO database, every new transaction would require scanning old blocks repeatedly to check whether the referenced output still exists and is unspent.

That would be very inefficient.

What does it store?

For each currently unspent output, it stores things like:

- transaction ID
- output index
- value
- locking script

Only unspent outputs are kept there.

Spent outputs are removed.

How is it used in validation?

When a new transaction arrives, the node checks:

1. Does each referenced parent UTXO exist in the DB?
2. Is it still unspent?
3. Does the unlocking proof match the owner condition?
4. Is total input value at least total output value?

If yes:

- remove old parent UTXOs from DB
 - add new child outputs to DB
-

Simplified validation algorithm

You can remember it like this:

Step 1

Check all inputs exist in UTXO DB

Step 2

Check signatures / ownership conditions

Step 3

Check input sum $\geq$ output sum

Step 4

If valid, remove spent inputs and add new outputs

That is the whole state update idea.

What sir can ask

Sir can ask:

- what is UTXO database
 - why Bitcoin needs chainstate
 - what happens to DB after valid transaction
 - explain validation algorithm
 - simulate whether a transaction is valid from given DB values
-

6) Transaction Fee and Fee Rate

What is transaction fee?

Transaction fee is the difference between:

total input value – total output value

That leftover is not assigned to any output, so it becomes miner fee.

Example

If transaction input total = 1.5 BTC

and outputs total = 1.499 BTC

then fee = 0.001 BTC

Why does Bitcoin have transaction fees?

Fees are paid to miners as incentive to include the transaction in a block.

Since block space is limited, users compete by offering fees.

Important idea: fee does NOT depend on transaction value

This is one of the biggest Week 7 concepts.

A \$1 payment can pay the same fee as a \$100,000 payment if the transaction size in bytes is the same.

Why?

Because miners care about **space**, not dollar meaning.

Fee rate

Fee rate means:

fee per byte

or more commonly in modern language:

fee per vbyte / sat per byte-like unit

In the exam, if you use simple size-based reasoning, think of it as:

- bigger transaction size → higher fee needed
-

Why transaction size matters

A transaction with:

- more inputs
- more outputs
- more script data

takes more bytes.

More bytes means it occupies more block space.

So higher-size transactions often need higher total fee.

Micropayment problem

This is why coffee payments are hard directly on-chain.

Even if the payment value is tiny, the byte size may still be significant, so the fee may be too large compared to the payment amount.

What sir can ask

Sir can ask:

- calculate transaction fee
 - explain fee vs fee rate
 - explain why fee depends on size not value
 - compare micropayments with large payments under same byte size
 - explain why forgetting change output makes huge fee
-

7) Bitcoin Transaction Types

Now the last major topic group.

7.1 P2PKH — Pay to Public Key Hash

This is the classic legacy transaction type.

The output is locked to the **hash of the recipient's public key**.

To spend it later, the spender must provide:

- public key
- valid signature

And the script checks:

- does provided pubkey hash match stored pubkey hash?
- does signature verify?

This is the standard “normal Bitcoin payment” type in older transactions.

7.2 P2MS — Pay to Multisig

This type directly locks an output to multiple public keys and requires **M-of-N** signatures.

Example:

- 2-of-3 multisig

That means any 2 out of the 3 authorized keys can spend the output.

This is useful for:

- shared accounts
- escrow
- joint control
- advanced protocols

It is an older multisig style.

7.3 P2SH — Pay to Script Hash

Instead of locking directly to a public key hash, P2SH locks to the **hash of an entire script**.

That means the output says:

“To spend me, reveal the script whose hash matches this, and satisfy that script.”

This is useful for complex spending conditions because the blockchain output stores only the script hash, not the whole long script.

So P2SH hides complexity until spending time.

7.4 P2WPKH — Pay to Witness Public Key Hash

This is the SegWit version of P2PKH.

It still means payment to a public key hash, but:

- `scriptSig` is empty
- signature and public key go into the **witness**
- txid excludes witness, helping fix malleability

So P2WPKH is like a cleaner SegWit-era version of normal key-hash payment.

7.5 P2WSH — Pay to Witness Script Hash

This is the SegWit version of P2SH.

Instead of storing script in old style, it stores the **hash of a witness script**.

Later, the witness reveals:

- the real script
- the required unlocking data

This is commonly used for:

- multisig
- more complex script conditions

So P2WSH is the SegWit version of script-hash locking.

7.6 P2TR — Pay to Taproot

This is the Taproot-era output type.

At your current lecture level, you mainly need to know it is a newer Bitcoin transaction type under **SegWit v1**, designed to support more advanced spending conditions more efficiently and privately.

It improves flexibility and can make complex spending appear simpler on-chain.

For your exam, if sir includes it in a table/objective question, remember:

- newer format
- Taproot
- advanced / more efficient / more private script handling

Quick comparison of transaction types

P2PKH

- lock to public key hash
- spender reveals pubkey + signature
- legacy normal payment

P2MS

- lock directly to multiple pubkeys
- needs M-of-N signatures
- multisig

P2SH

- lock to hash of script
- actual script revealed later
- useful for complex policies

P2WPKH

- SegWit version of P2PKH
- witness carries signature/pubkey
- fixes malleability issue better

P2WSH

- SegWit version of P2SH
- lock to hash of witness script
- used for complex multisig/script policies

P2TR

- Taproot
- newer, more efficient and privacy-improving structure
- SegWit v1 generation

Best way to prepare from these topics

Since sir said the paper has:

- conceptual
- simulation/numerical
- objectives/table fill

here is how you should think:

Conceptual topics

- Merkle tree and proof
- UTXO database
- transaction types
- malleability / SegWit ideas if linked
- mining concept

Simulation / numerical topics

- Merkle root construction
- Merkle proof verification
- mining with target and nonce
- transaction fee calculation
- UTXO validation / unlocking
- signature verification flow
- identifying valid vs invalid child transaction

Objective / table fill

Very likely:

- compare transaction types
 - match P2PKH / P2SH / P2WPKH / P2WSH / P2TR / P2MS
 - maybe fee/rate or UTXO vs account ideas
-

Final short list of all topics sir mentioned

Here they are cleanly again:

- Merkle Tree
- Merkle Proof
- Bitcoin transaction signature construction
- Bitcoin transaction signature verification
- Bitcoin UTXO unlocking
- Proof-of-Work
- Mining
- Target hash
- Nonce
- UTXO Database
- Transaction Fee
- Transaction Rate / Fee Rate
- P2PKH
- P2MS
- P2SH
- P2WPKH
- P2WSH
- P2TR